DNA Sequence Matching using Parallel Computing (CPU vs GPU Performance Analysis)

USE CASE SCENARIO

```
        Scenario  :   Crime Investigation in a Village
```

In a village, a murder has occurred. During investigation, police collect:

A hair sample from the crime scene → used to extract DNA (target DNA) DNA samples from all suspects (100–10,000+ people) in the village

All these DNA samples are stored in a file (Human.txt)

Problem

bold text Find which person's DNA matches the DNA found at the crime scene.

We compare the crime DNA with all people's DNA using three methods. In the serial CPU, DNA is checked one by one, so it is slow. In the parallel CPU, multiple cores check many DNA samples at the same time, making it faster. In the GPU, thousands of threads compare many DNA sequences simultaneously, making it the fastest. This is important because real DNA databases can be very large. Faster processing helps identify suspects quickly in real-world investigations.

Why This Is Important .

Real DNA databases have very large data (lakhs or millions of people) Faster checking helps catch the criminal quickly.

```
%%writefile generate_dataset.py
import random

dna_chars = ['A', 'T', 'C', 'G']

num_people = 50000
dna_length = 800

with open("Human.txt", "w") as f:
    for i in range(1, num_people + 1):
        dna = ''.join(random.choice(dna_chars) for _ in range(dna_length))
        f.write(f"Person{i} {dna}\n")

print("Dataset generated")
```

```
Overwriting generate_dataset.py
```

```
!python generate_dataset.py
```

```
Dataset generated
```

```
from google.colab import files
files.download("Human.txt")
```

CPU CODE (Serial vs Parallel)

```
%%writefile cpu_dna.c
#include <stdio.h>
#include <string.h>
#include <time.h>
#include <omp.h>

#define MAX 50000
#define LEN 1000

char names[MAX][50];
char dna[MAX][LEN];
char crime_dna[LEN];

int compareDNA(char *a, char *b) {
    for (int i = 0; a[i] != '\0'; i++) {
```

```
        if (a[i] != b[i]) return 0;
    }
    return 1;
}

int main() {
    FILE *fp = fopen("Human.txt", "r");
    int n = 0;

    while (fscanf(fp, "%s %s", names[n], dna[n]) != EOF) n++;
    fclose(fp);

    printf("Total Records: %d\n", n);

    // USER INPUT
    printf("Enter Crime DNA: ");
    scanf("%s", crime_dna);

    int repeat;
    printf("Enter repeat count: ");
    scanf("%d", &repeat);

    // SERIAL
    clock_t s1 = clock();
    for (int r = 0; r < repeat; r++)
        for (int i = 0; i < n; i++)
            compareDNA(dna[i], crime_dna);
    clock_t e1 = clock();

    double serial = (double)(e1 - s1) / CLOCKS_PER_SEC;

    // PARALLEL
    double s2 = omp_get_wtime();
    for (int r = 0; r < repeat; r++) {
        #pragma omp parallel for schedule(static)
        for (int i = 0; i < n; i++)
            compareDNA(dna[i], crime_dna);
    }
    double e2 = omp_get_wtime();

    double parallel = e2 - s2;

    printf("Serial = %f sec\n", serial);
    printf("Parallel = %f sec\n", parallel);

    // SAVE
    FILE *out = fopen("cpu_output.txt", "w");
    fprintf(out, "%f %f", serial, parallel);
    fclose(out);

    return 0;
}
```

```
Overwriting cpu_dna.c
```

```
!gcc -fopenmp cpu_dna.c -o cpu

print("Enter the repeat count as the 'how many times loop runs'")
print("Beter to give the 10000 ten thousand times iteratons ")
!./cpu
```

```
Enter the repeat count as the 'how many times loop runs'
Beter to give the 10000 ten thousand times iteratons
Total Records: 50000
Enter Crime DNA: GGTCCAGCTCGAACCAGTTTTTCTGATCTGTGGATCTCAACTATCTCACGGAACTAAGTGCACTTCATAATGGACTCTCGTTCCCTGCAGTACAACCCCTTCTCTCT
Enter repeat count: 1000
Serial = 0.774974 sec
Parallel = 0.554812 sec
```

GPU CODE (CUDA)

```
%%writefile gpu_dna.cu
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <cuda.h>

#define MAX 50000
```

```
#define LEN 1000

__global__ void matchDNA(char *dna, char *crime, int *res, int n, int len) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    if (i >= n) return;

    int match = 1;
    for (int j = 0; j < len; j++) {
        if (dna[i * len + j] != crime[j]) {
            match = 0;
            break;
        }
    }
    res[i] = match;
}

int main() {
    FILE *fp = fopen("Human.txt", "r");

    static char dna[MAX][LEN];
    char crime[LEN];

    int n = 0;
    while (fscanf(fp, "%*s %s", dna[n]) != EOF) n++;
    fclose(fp);

    printf("Total Records: %d\n", n);

    // USER INPUT
    printf("Enter Crime DNA: ");
    scanf("%s", crime);

    int len = strlen(crime);

    char *flat = (char*)malloc(n * LEN);
    for (int i = 0; i < n; i++)
        memcpy(&flat[i*LEN], dna[i], LEN);

    char *d_dna, *d_crime;
    int *d_res;

    cudaMalloc(&d_dna, n*LEN);
    cudaMalloc(&d_crime, LEN);
    cudaMalloc(&d_res, n*sizeof(int));

    cudaMemcpy(d_dna, flat, n*LEN, cudaMemcpyHostToDevice);
    cudaMemcpy(d_crime, crime, LEN, cudaMemcpyHostToDevice);

    int threads = 256;
    int blocks = (n + threads - 1)/threads;

    cudaEvent_t s, e;
    cudaEventCreate(&s);
    cudaEventCreate(&e);

    cudaEventRecord(s);

    matchDNA<<<blocks, threads>>>(d_dna, d_crime, d_res, n, len);
    cudaDeviceSynchronize();

    cudaEventRecord(e);
    cudaEventSynchronize(e);

    float ms;
    cudaEventElapsedTime(&ms, s, e);

    printf("GPU = %f ms\n", ms);

    FILE *out = fopen("gpu_output.txt", "w");
    fprintf(out, "%f", ms/1000.0);
    fclose(out);

    return 0;
}
```

```
Writing gpu_dna.cu
```

```
!nvcc gpu_dna.cu -o gpu
!./gpu
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future
Total Records: 50000
```

```
Enter Crime DNA: GGTCCAGCTCGAACCAGTTTTTCTGATCTGTGGATCTCAACTATCTCACGGAACTAAGTGCACTTCATAATGGACTCTCGTTCCCTGCAGTACAACCCCTTCTCTCT
GPU = 100.037666 ms
```

PYTHON CODE (Visualization + Details)

```
import matplotlib.pyplot as plt

with open("cpu_output.txt") as f:
    serial, parallel = map(float, f.read().split())

with open("gpu_output.txt") as f:
    gpu = float(f.read())

print("\nFINAL RESULT")
print("Serial   :", serial)
print("Parallel :", parallel)
print("GPU      :", gpu)

methods = ['Serial', 'Parallel', 'GPU']
times = [serial, parallel, gpu]

plt.figure()
plt.bar(methods, times)

for i, v in enumerate(times):
    plt.text(i, v, f"{v:.4f}", ha='center')

plt.title("Performance Comparison")
plt.ylabel("Time (sec)")
plt.show()
```

```
FINAL RESULT
Serial   : 0.774974
Parallel : 0.554812
GPU      : 0.100038
```